

Throughput-Optimized Service Routing for Microservice Flows in LEO Satellite Networks

Xindi He , Graduate Student Member, IEEE, Ting Wang , Senior Member, IEEE,
Yuanming Shi , Senior Member, IEEE, and Xin Liu , Member, IEEE

Abstract—Satellite-based microservice systems have emerged as a promising architecture for enabling scalable and flexible service deployment in Low Earth Orbit (LEO) satellite networks, which are increasingly recognized as a key solution to meet the rising demand for global communication and computation, especially in remote and underserved regions. However, routing microservices efficiently in such systems presents major challenges due to the dynamic topology, intermittent connectivity, and unstable link conditions inherent to satellite constellations. These issues become even more severe under high traffic loads. To address these challenges, we propose Service Pressure, a novel routing algorithm specifically designed for satellite-based microservice systems. Service Pressure comprises two key components: first, the construction of an augmented subgraph to model the complex execution and data transmission dependencies of microservices; second, a distributed service routing algorithm that utilizes queue backlogs. This combination enables the algorithm to effectively handle high throughput and adapt to the dynamic network conditions of satellite constellations. By optimizing resource utilization, minimizing latency, and balancing load across satellite nodes, Service Pressure ensures efficient and stable service orchestration even under fluctuating traffic conditions. Extensive simulations demonstrate that our approach significantly outperforms existing routing methods, particularly in terms of throughput, latency, and stability. Service Pressure offers a significant advancement in satellite microservice routing, making it ideal for next-generation space-ground integrated networks.

Index Terms—Microservice, routing, satellite networks, backpressure, Low Earth Orbit (LEO).

I. INTRODUCTION

THE rapid evolution of communication networks from 5G to emerging 6G systems has brought a series of new

challenges to traditional terrestrial infrastructure, such as a significant increase in traffic and ensuring seamless coverage, especially in remote and underserved areas. This highlights a critical gap in global connectivity, as terrestrial networks alone are unable to provide ubiquitous services with stable performance. To address these challenges, satellite networks, especially those based on low-Earth orbit (LEO) constellations, are increasingly seen as a viable solution. Equipped with advanced inter-satellite links (ISLs) and high-performance payloads [1], LEO satellite constellations provide a powerful and scalable infrastructure to complement and extend terrestrial mobile communication systems, facilitating the widespread coverage required for next-generation networks such as 6G [2], [3]. Hundreds of LEO satellite computing nodes together constitute the Satellite Computing and Processing Network (SCPNs) [3]. In the context of 6G space-ground integrated networks, how to strategically allocate communication and computing resources to efficiently meet diverse intelligent service requests has become a key challenge [4], [5], [6].

Traditionally, monolithic applications and their functional modules were predominantly deployed in terrestrial data centers to process data sent back from satellites. However, with the development of containerization, these monolithic applications can now be decomposed into stateless microservices [7]. These microservices are packaged into isolated containers, which can be independently deployed, managed, and scaled [8]. By combining multiple microservices in different configurations, this modular approach allows for greater flexibility in meeting diverse, dynamic demands. Moreover, utilizing the distributed computing capabilities of satellite constellations enables these microservices to be dynamically scheduled across the computing nodes of SCPNs [9], [10], [11]. This not only facilitates efficient, on-board computation but also drastically reduces transmission delays caused by the limited bandwidth and challenging communication conditions between space and terrestrial networks, optimizing resource utilization across the entire satellite system.

However, integrating microservices into satellite networks presents several significant challenges. One of the primary difficulties stems from the multifaceted dynamic nature of these constellations, which includes both predictable and stochastic elements. While the topological evolution caused by satellites' orbital mechanics is deterministic and known in advance, it still results in continuous, high-frequency changes in the connectivity graph [12]. Compounding this predictable dynamism are unpredictable link failures or satellite downtime, stemming

Received 6 May 2025; revised 14 October 2025; accepted 1 December 2025. Date of publication 4 December 2025; date of current version 6 April 2026. The work of Ting Wang was supported in part by the Key Program of National Natural Science Foundation of China under Grant 62432007, and in part by the Natural Science Foundation of Shanghai under Grant 25ZR1401104. The work of Yuanming Shi was supported in part by the National Natural Science Foundation of China under Grant 62522117 and Grant 62271318, and Yangtze River Delta Science and Technology Innovation Community Joint Research (Basic Research) Project under Grant BK 2024CSJZN00303. The work of Xin Liu was supported in part by the National Natural Science Foundation of China under Grant 62302305. Recommended for acceptance by C. M. Pinotti. (Corresponding author: Ting Wang.)

Xindi He and Ting Wang are with the Shanghai Key Lab. of Trustworthy Computing, MoE Engineering Research Center of Software/Hardware Co-Design Technology and Application, East China Normal University, Shanghai 200062, China (e-mail: 51275902046@stu.ecnu.edu.cn; twang@sei.ecnu.edu.cn).

Yuanming Shi and Xin Liu are with the School of Information Science and Technology, ShanghaiTech University, Shanghai 201210, China (e-mail: shiym@shanghaitech.edu.cn; liuxin7@shanghaitech.edu.cn).

Digital Object Identifier 10.1109/TMC.2025.3640395

from the inherent vulnerabilities of space-based hardware, limited power budgets forcing temporary shutdowns, or other unforeseen faults [13]. The combination of these factors makes it difficult to maintain stable and efficient communication. Furthermore, the limited bandwidth and high latency associated with inter-satellite communication exacerbate these challenges, especially when large volumes of data are transmitted across vast distances [14]. In addition, satellite networks are inherently distributed, with resources spread across multiple satellites in a constellation. This distributed nature necessitates efficient coordination and management of computational resources to ensure that the system can meet the diverse demands of applications in real-time. Traditional centralized routing algorithms [15], [16], [17], which rely on static topologies and global knowledge of the network, are not suitable for such dynamic environments. These algorithms often struggle to handle the variability and unpredictability of satellite networks, leading to suboptimal performance in terms of both latency and resource utilization. Backpressure-based routing has been widely adopted as a powerful, distributed paradigm for effectively managing dynamic topologies and optimizing throughput, demonstrating its feasibility in a wide range of mobile multi-hop networks [18], [19], [20]. However, a well-known drawback of conventional backpressure algorithms is their tendency to cause significant end-to-end latency, as their forwarding decisions without explicit path computation can lead to unnecessarily long paths.

To address these challenges, we propose Service Pressure, a novel service routing algorithm designed specifically for satellite microservice systems. Our algorithm takes into account the unique characteristics of satellite networks, such as their dynamic topology, distributed nature, and the need to handle high traffic loads. By dynamically building an Augmented Satellite-Microservice Flow Sub-Graph and running a backpressure algorithm with path optimization on it, our algorithm ensures that service requests are routed efficiently. This adaptability enables the algorithm to minimize latency, balance load between satellite nodes, and maximize the use of available resources. In addition, the algorithm is able to respond to fluctuations in traffic and network conditions, making it well suited for the high-demand, real-time requirements of satellite microservice systems. With these advances, our algorithm addresses the urgent need for efficient service orchestration in next-generation satellite communication and processing networks.

Our main contributions are summarized as follows:

- *Microservice Flow Modeling in SCPN*: We have completed the modeling of microservice flows within SCPNs. This model captures the dependencies and execution order of microservices, allowing for a detailed representation of their communication and computational requirements in the satellite network. By accurately mapping these flows, our model provides a foundation for optimizing service routing and resource allocation, ensuring efficient operation within the dynamic and distributed SCPN environment.
- *Augmented Satellite-Microservice Subgraph Construction*: We propose an innovative method to construct the Augmented Satellite-Microservice Subgraph. This

subgraph abstracts the execution and data transmission processes of microservices across different service flows in the satellite network into a unified graph structure. The purpose of this construction is to ensure that any feasible service routing scheme of each flow corresponds to a connected subgraph of the augmented graph. This abstraction provides a clear, operational mathematical model for complex resource allocation and communication path selection, facilitating subsequent optimization algorithms.

- *Distributed Service Routing Algorithm Based On Backpressure*: We develop a distributed service routing method based on backpressure, along with path optimization. In this method, each node only needs to consider its neighboring satellites, which reduces computational complexity and effectively handles the dynamic topology of LEO satellite networks. The algorithm ensures efficient resource allocation for high-priority microservice flows by flexibly adjusting weights. Extensive experiments demonstrate that our algorithm significantly outperforms other routing methods in terms of throughput and latency, while maintaining excellent stability even under complex traffic conditions.

II. RELATED WORK

SCPNs represent a typical example of a discrete distributed computing architecture. Extensive research has already been conducted on decomposing applications, traditionally reliant on ground-based data centers, into microservices that can be deployed across satellite systems [21], [22]. These microservices are inherently independent, scalable, reusable, and stateless. A substantial body of work exists on the routing and scheduling of microservices in traditional terrestrial distributed systems [23], [24], and some studies have extended these concepts to mobile edge computing environments [25], [26]. However, in satellite-based microservice systems, the high latency and limited bandwidth between satellites contrast sharply with the low-latency, high-bandwidth communication environments of terrestrial data centers. Additionally, the dynamic nature of satellite orbits and the intermittent connections between satellites add further complexity to maintaining stable communication links. These challenges render traditional deterministic, global routing and scheduling strategies impractical for satellite systems. Consequently, there is a critical need for the development of novel routing and scheduling algorithms tailored specifically to satellite networks, which can leverage real-time feedback, adaptive scheduling, and distributed resource management.

Extensive research has focused on the challenge of efficient end-to-end data routing in LEO satellite networks, spawning a variety of algorithms [15], [16], [17], [27], [28], [29]. However, these algorithms are primarily designed for pure data transmission, that is, finding the optimal path between a single source and a single destination. They cannot inherently address the unique complexities introduced by microservice applications. At present, there remains a significant gap in research specifically focused on scheduling and routing schemes for microservices within satellite networks. In [30], the authors proposed a framework for edge microservice scheduling in satellite-ground

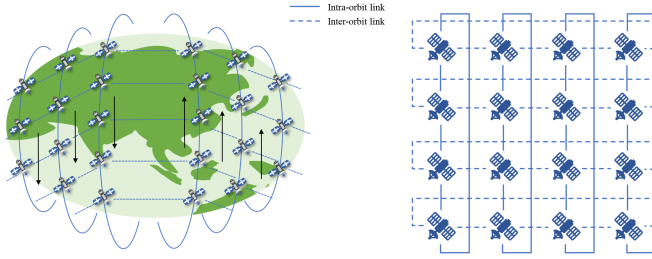


Fig. 1. LEO satellite network and its 2D topology.

hybrid networks, which optimized task scheduling through dynamic link adjustments and minimized the end-to-end delay of tasks. Similarly, in [31], the authors introduced the SatEdge platform, a satellite-based edge cloud solution designed to leverage satellite-provided edge computing services to enhance system reliability, global coverage, and low-latency communications. However, these studies do not provide targeted optimizations for service routing and the load balancing of inter-satellite links. In [15], the authors have focused on optimizing service request routing in satellite communication networks, but they do not incorporate multiple sets of different data flows into their analysis.

Backpressure-based routing has been well-established in terrestrial distributed networks, where it is renowned for achieving maximum throughput under ideal conditions. These algorithms dynamically adjust traffic to avoid congestion, ensuring optimal resource utilization. Some researchers have attempted to apply backpressure routing algorithms to satellite communication networks with promising results [32], [33], [34]. However, none of these works have considered the integration of backpressure routing with microservice architectures in satellite systems.

III. SYSTEM MODEL

A. LEO Satellite Network Model

We consider a typical LEO satellite network in which satellites communicate through laser-based ISLs. In this network topology model, each satellite in the LEO constellation is assumed to be equipped with four ISLs: two inter-orbit links connect to satellites in adjacent orbits, while the remaining two intra-orbit links connect to satellites within the same orbit. To enable communication between non-adjacent satellites, a multi-hop relaying mechanism is employed, where data is routed across multiple intermediate satellites. The 2D topology illustration is shown in Fig. 1.

The satellite network is modeled as a time-slotted network, where each time slot represents a short period during which the LEO constellation's topology remains static. The satellite network topology at time slot t is represented as an undirected graph $G(t) = (V(t), E(t))$. $V(t) \subseteq V$ represents the satellites available at time slot t , and $E(t)$ represents the set of feasible communication links between satellites at time slot t , where an edge $e_{i,j} \in E(t)$ signifies an ISL link between satellites v_i and v_j .

B. Satellite-Microservice Colocation Model

Consider a set of microservices $M = \{m_1, m_2, \dots, m_{|M|}\}$, with the deployment strategy on the LEO constellation denoted as $D = \{D_{v_i} \neq \emptyset \mid \forall v_i \in V(t)\}$, where D_{v_i} represents the set of microservices deployed on satellite v_i . Due to the limited computational, storage, and power resources on each satellite, only a minimal subset of deployable microservices can typically be hosted on a single satellite. We assume that each microservice can have up to N_ν replicas distributed across the same or different satellites, and each satellite will host at least one but no more than N_ℓ microservices. This constraint reflects the resource limitations of each satellite, meaning that for all microservices deployed on satellite v_i , the set is represented as $D_{v_i} = \{m_k \mid k \in [1, |M|]\}$ and satisfies $1 \leq |D_{v_i}| \leq N_\ell$. Similarly, the set of satellites hosting microservice m_i is denoted by $V_{m_i} = \{v_l \mid l \in [1, |V|]\}$, satisfying $1 \leq |V_{m_i}| \leq N_\nu$. The microservice deployment is established during the initialization phase of the LEO satellite network topology and remains static, even if the network topology evolves due to satellite movement.

C. Satellite-Microservice Flow Model

Consider a set of applications $N = \{n_1, n_2, \dots, n_{|N|}\}$, where each application $n_i \in N$ consists of a subset of microservices from the set M , denoted as $n_i = \{m_j, \dots, m_k\}$. The execution of an application n_i is determined by the dependency relationships among its microservices. These dependencies can form either a chain or a tree structure, where no cyclic dependencies (rings) are allowed. We define $G(n_i) = (M_i, W_i)$ as the dependency graph for application n_i , where $M_i \subseteq M$ is the set of microservices involved in n_i , and $W_i \subseteq M_i \times M_i$ represents the dependency relationships. If $(m_j, m_k) \in W_i$, it indicates that the execution of microservice m_{i_j} must precede the execution of m_{i_k} . These dependencies satisfy transitivity, meaning that if $(m_j, m_k) \in W_i$ and $(m_k, m_l) \in W_i$, then $(m_j, m_l) \in W_i$.

The execution of application n_i is abstracted as a unique microservice flow f_i , which defines the order of execution for the microservices in M_i based on their dependency graph $G(n_i)$. Unlike a strict linear order, the microservice flow f_i allows parallel execution of microservices that are not directly or transitively dependent on one another. For example, if microservices m_k and m_l both depend on m_j but have no dependency between them, they can execute in parallel after m_j is completed.

For the entire system, the set of all microservice flows is denoted as $F = \{f_1, f_2, \dots, f_{|N|}\}$, where each f_i corresponds to a unique application $n_i \in N$. When a ground-based user sends a service request to invoke an application n_i , the system designates a source satellite $v_{src} \in V$ to receive the request from the ground station. The execution of the application is then carried out across the LEO satellite network according to the microservice flow f_i , with each microservice $m_j \in M_i$ being executed on a specific satellite. The final results are transmitted to a designated destination satellite $v_{dst} \in V$, which coordinates the overall output.

For simplicity, we define that a single service request sent by a user to the source satellite can invoke only one application. If a user needs to invoke multiple applications, multiple service

requests must be issued to the source satellite. Thus, each user service request represents a complete invocation of a specific application $n_i = \{m_j, m_{j+1}, \dots, m_{j+k}\}$, which corresponds to a complete traversal of a microservice flow $f_i \in F$. In this process, data packets travel from the source satellite v_{src} , sequentially passing through $v_1 \in V_{m_j}, v_2 \in V_{m_{j+1}}, \dots, v_{k+1} \in V_{m_{j+k}}$, along with any necessary multi-hop relay satellites, until they reach the destination satellite v_{dst} for final processing.

D. Satellite-Microservice Network Capacity Model

For each application $n_i \in \mathcal{N}$, the maximum sustainable request generation rate can be defined as λ_i , representing the highest number of requests (measured in data packets) that the system can handle per unit time without causing unbounded queue growth. Assuming that data packets are generated according to a Poisson process, and disregarding any potential loss during transmission from the ground to the LEO satellite network, we define an arrival rate matrix Λ to represent the arrival rates of all microservice flows at each satellite in the LEO satellite network as follows:

$$\Lambda = \begin{bmatrix} \lambda_{1,v_1} & \lambda_{1,v_2} & \dots & \lambda_{1,v_{|V|}} \\ \lambda_{2,v_1} & \lambda_{2,v_2} & \dots & \lambda_{2,v_{|V|}} \\ \vdots & \vdots & \ddots & \vdots \\ \lambda_{|N|,v_1} & \lambda_{|N|,v_2} & \dots & \lambda_{|N|,v_{|V|}} \end{bmatrix}$$

Here, $\lambda_{i,v}$ denotes the arrival rate of requests for application n_i at satellite v . Since only the source satellite receives service requests directly from ground stations, the arrival rates at other satellite nodes in the matrix are assumed to be zero. Consequently, we can simplify the analysis by considering only the arrival rate vector $\lambda_{src} = (\lambda_{1,v_{src}}, \lambda_{2,v_{src}}, \dots, \lambda_{|N|,v_{src}})$, where v_{src} represents the source satellite responsible for receiving requests for application n_i .

Assume that all data packets are of the same size. During the execution of microservices, if the dependency structure specifies that multiple downstream microservices must be executed in parallel, data packets will be replicated into multiple copies and forwarded to the corresponding satellites hosting these downstream microservices. Specifically, for a data packet passing through satellite $v_i \in V(m_k)$ within microservice flow f_j , the following rules apply:

- 1) Processing at Satellite v_i : If the packet belongs to microservice flow f_j and needs to be processed by microservice m_k according to the dependency of application n_j , then satellite v_i will process the packet. After processing, if the next step in the dependency graph supports multiple microservices to execute in parallel (for example, $m_k \rightarrow \{m_o, m_p, m_q\}$), the packet will be replicated into separate copies, each of which is forwarded to the corresponding satellites hosting m_o, m_p and m_q .
- 2) Forwarding at Intermediate Satellite v_i : If satellite v_i is not responsible for processing a packet, it will act as an intermediate relay node. In this case, the packet will be forwarded directly to the next satellite according to the routing policy.

At time slot t , all data packets arriving at satellite v_i are added to its backlog queue. We define the computational throughput of satellite v_i as c_{v_i} , which represents the maximum number of data packets the satellite can process within a single time slot. If the length of the backlog queue exceeds the satellite's computational throughput, the excess packets will remain in the queue and wait for future processing. Similarly, we define the maximum transmission throughput of the ISL between satellites v_a and v_b as l_{v_a,v_b} , which represents the maximum number of data packets that satellite v_a can transmit to v_b within a single time slot. If the total number of data packets requiring forwarding from v_a to v_b in its backlog queue exceeds the transmission throughput l_{v_a,v_b} , the excess packets will also remain in the queue and wait for future transmission. The backlog queue of satellite v_i , containing data packets for microservice flow $f_j \in F$, is denoted as $Q_{f_j}^{v_i}(t)$, and it satisfies the following equation:

$$|Q_{f_j}^{v_i}(t+1)| = |Q_{f_j}^{v_i}(t)| - \sum_{e_{i,b} \in E(t)} \mu_{(v_i,v_b)}^{f_j}(t) + \sum_{e_{a,i} \in E(t)} \mu_{(v_a,v_i)}^{f_j}(t) + A_{v_i}^{f_j}(t), \quad (1)$$

where $\mu_{(u,v)}^{f_j}(t)$ denotes the actual data packet transmission volume from satellite u to satellite v over the ISL within microservice flow f_j . $A_{v_i}^{f_j}(t)$ represents the arrival rate of data packets for microservice flow f_j at satellite v_i . For the source satellite v_{src} , this value equals the number of data packets received from the ground station at time slot t . For non-source satellites, $A_{v_i}^{f_j}(t)$ denotes the extra packet copies generated due to the parallel execution of dependent microservices. This is determined by the number of microservices that need to be executed in parallel, which is directly related to the number of independent microservices scheduled for execution after the current one, based on the application's dependency graph.

Our objective is to maximize the aggregate source arrival rates λ_{src} that the system can stably support, while maintaining low latency and balanced resource utilization across the satellite network. The algorithm needs to be flexible, allowing for the dynamic adjustment of priorities for different applications based on their specific needs. By stabilizing backlog queues, prioritizing packet forwarding, and efficiently allocating resources, the system ensures that service requests are handled according to their assigned priority. Using a distributed and adaptive approach, the algorithm can adjust to varying traffic conditions, balance resource usage, and reduce delays for higher-priority microservice flows. This enables the system to handle higher aggregate arrival rates while maintaining low latency, especially for high-priority applications. This motivates the design of the Service Pressure Algorithm, which we introduce in the next section.

IV. SERVICE PRESSURE ALGORITHM

In this section, we introduce Service Pressure, which is an efficient dynamic service routing framework aiming to maximize service request throughput while minimizing request latency. We first detail our augmented satellite-microservice connected

sub-graph construction algorithm, and then elaborate on the design of our backpressure-based service routing algorithm.

A. Augmented Satellite-Microservice Flow Sub-Graph Construction

For each request from the ground station at time slot t , we construct the augmented satellite-microservice flow sub-graph $G_A^i(t) = (V_A^i(t), E_A^i(t))$ for its corresponding f_i . This is a directed graph with $V_A^i(t) = \emptyset$ and $E_A^i(t) = \emptyset$ initially. Then, the process of constructing $G_A^i(t)$ is detailed as follows:

Step 1: This step involves adding the virtual source node v_{src^T} and the virtual destination node v_{dst^T} into the augmented vertex set $V_A^i(t)$.

Step 2: Considering the dependency graph $G(n_i) = (M_i, W_i)$, for each microservice $m_j \in M_i$ and each satellite $v_k \in V_{m_j}$, this step creates two virtual vertices: v_{k^C} and v_{k^T} . These virtual vertices, called virtual computing nodes and virtual transmission nodes, represent the abstraction of the computing power and transmission capacity of the satellite v_k , respectively. A virtual directed edge $v_{k^C} \rightarrow v_{k^T}$ is added, with a capacity of c_{v_k} . This edge abstracts the process of the satellite utilizing its computing resources to process microservice data packets. Both the virtual vertices and the edge are added to the augmented vertex set $V_A^i(t)$ and edge set $E_A^i(t)$.

Step 3: For the dependency relationship between two microservices m_j and m_k , that is, $\forall(m_j, m_k) \in W_i$, this step identifies all satellites hosting m_j and m_k . For each pair of satellites $v_p \in V_{m_j}$ and $v_q \in V_{m_k}$, if v_p and v_q are directly connected (i.e., no relay satellites are required), a directed edge $v_{p^T} \rightarrow v_{q^C}$ is created. This edge abstracts the process of satellite v_p using its transmission resources to transmit microservice data packets to satellite v_q . Then set the capacity of this edge:

$$c(v_{p^T}, v_{q^C}) = \begin{cases} \infty, & \text{if } v_p = v_q, \\ l_{v_p, v_q}, & \text{if } v_p \neq v_q, \end{cases}$$

where l_{v_p, v_q} denotes the maximum transmission capacity of the ISL between v_p and v_q .

If v_p and v_q are not directly connected, this step searches for available relay satellites in $V(t)$ and constructs all possible relay paths between v_p and v_q . For each relay path, the actual relay satellites involved are represented by their virtual transmission nodes. Suppose the relay path consists of t relay satellites $\{v_{r_1}, v_{r_2}, \dots, v_{r_n}\}$, where n is the number of hops in the path. These relay satellites' virtual transmission nodes are added to the augmented vertex set $V_A^i(t)$. Directed edges $v_{p^T} \rightarrow v_{r_1^T}, v_{r_1^T} \rightarrow v_{r_2^T}, \dots, v_{r_{n-1}^T} \rightarrow v_{r_n^T}, v_{r_n^T} \rightarrow v_{q^C}$ are then added to the augmented edge set $E_A^i(t)$ to represent data packet transmission along the relay path. Each edge is assigned a capacity equal to the transmission capacity of the corresponding ISL it represents.

Step 4: To complete the augmented graph, this step connects the source node v_{src^T} and the destination node v_{dst^T} to the other nodes in the graph. Specifically, m_{src} is defined as the unique root node of the dependency graph $G(n_i) = (M_i, W_i)$, representing the first microservice in the execution sequence of application n_i . For v_{src^T} , directed edges are added to all satellites that host m_{src} . For each satellite $v_p \in V_{m_{src}}$, a directed edge $v_{src^T} \rightarrow v_{p^C}$ is created, representing the source sending requests to satellites hosting the first microservice. The capacity of this edge is set to ∞ , as the source node can send requests without computational constraints. Similarly, M_{dst} is defined as the set of all leaf nodes in $G(n_i)$, representing the final microservices in the dependency graph that have no outgoing edges. For v_{dst^T} , directed edges are added from all satellites that host any microservice $m_{dst} \in M_{dst}$. For each satellite $v_q \in V_{m_{dst}}$, a directed edge $v_{q^T} \rightarrow v_{dst^T}$ is created, representing the final transmission of results from the satellite to the destination. The capacity of this edge is also set to ∞ , as the destination node can receive results without transmission constraints. If v_{src^T} or v_{dst^T} is not directly connected to the respective satellites, the relay process described in Step 3 is applied.

The complete construction process of the augmented graph is shown in Fig. 2. The purpose of constructing this augmented graph is to abstract the microservice execution and data transmission in each different microservice flow in the satellite network into a unified graph structure. For each service request calling application n_i , any feasible service routing scheme must be a connected subgraph of the augmented graph G_A^i , that is, all data packets belonging to microservice flow f_i will only be propagated and processed in G_A^i . This augmented graph represents the complex dependencies of resource allocation and communication path selection in the satellite network, providing a clear and operational mathematical representation for subsequent optimization algorithms.

B. Service Pressure Algorithm

The Service Pressure Algorithm is designed to operate on the augmented graph G_A^i for each individual microservice flow f_i , leveraging the structure of the augmented graph to dynamically allocate resources and determine routing paths. This algorithm operates in a distributed manner, meaning that the decision-making and resource allocation processes are localized to each virtual node. The service pressure at each virtual node reflects the current load and routing priority for microservice packets at that specific node, allowing the network to adapt dynamically to varying conditions.

1) *Definition of Service Pressure*: To accurately compute the Service Pressure at a virtual node $v' \in V_A^i$ for each data packet in a microservice flow f_i , it is essential to consider the *next executable microservice* for that packet. We define a function $\text{Next}(f_i, m) = \{m' \mid m' \in \text{Succ}(m)\}$, which represents all the direct successors of microservice m in the dependency graph

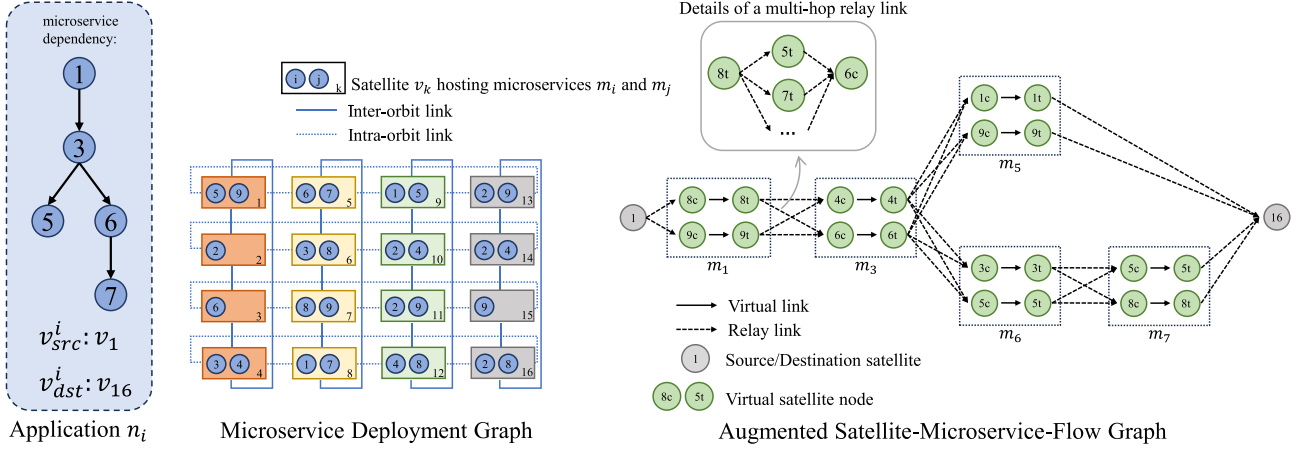


Fig. 2. Construction of the augmented graph.

$G(n_i)$. Let $p_{f_i}^{m_j}$ denote a data packet associated with microservice m_j that can be executed next based on the current execution progress in $G(n_i)$. Specifically:

- If the packet has most recently completed the execution of a microservice m_k , then $m_j \in \text{Next}(f_i, m_k)$.
- If the packet has completed all microservices in $G(n_i)$ and no further successors exist, we assign a special marker m_{end} to indicate that the packet has reached the end of its execution.

The set of packets classified by their next executable microservice is denoted as $\{p_{f_i}^{m_j}\}$, $m_j \in M_i \cup \{m_{end}\}$.

To enable fine-grained scheduling at each virtual node based on microservice dependencies, we first define the backlog queue $Q_{f_i}^v(t)$ at each actual satellite node v , which stores all packets of microservice flow f_i at time slot t . Each packet in $Q_{f_i}^v(t)$ is associated with a specific next executable microservice, according to the dependency structure of its application.

Building on this, we define the Service Pressure $\hat{Q}_{p_{f_i}}^{v' m_j}(t)$ at a virtual node v' as the number of packets in $Q_{f_i}^v(t)$ that have microservice m_j as their next executable step. Formally:

$$\hat{Q}_{p_{f_i}}^{v' m_j}(t) = \left| \left\{ p_{f_i}^{m_j} \mid p_{f_i}^{m_j} \in Q_{f_i}^v(t) \right\} \right|. \quad (2)$$

2) *Distributed Routing Algorithm With Path Optimization:* The algorithm is a backpressure-based algorithm that operates on each virtual node $v' \in V_A^i$ in all augmented graphs, which dynamically determines routing paths and allocates resources based on the largest differential backlog at each virtual node. The operation of the algorithm differs depending on whether the virtual node is a virtual computation node (c -type node) or a virtual transmission node (t -type node).

a) *Calculating the Largest Differential Backlog:* For each virtual node a , the algorithm calculates the largest differential backlog $\hat{w}_{(a,b)}$ on the edge (a, b) , where b represents one of the neighboring nodes of a . The computation of $\hat{w}_{(a,b)}$ differs based on the type of virtual node:

- *c-type node:* If the current node a is a c -type node (virtual computation node), the largest differential backlog $\hat{w}_{(a,b)}$

is computed as:

$$\hat{w}_{(a,b)}(t) = \max_{f_i \in F, m_j \in M_i} \left\{ \left(\hat{Q}_{p_{f_i}}^{a m_j}(t) \right)^{\alpha_{f_i}} - \sum_{m_k \in \text{Next}(f_i, m_j)} \left(\hat{Q}_{p_{f_i}}^{b m_k}(t) \right)^{\alpha_{f_i}} \right\}, \quad (3)$$

where $\hat{Q}_{p_{f_i}}^{a m_j}(t)$ is the Service Pressure for flow f_i associated with microservice m_j at node a , and $\hat{Q}_{p_{f_i}}^{b m_k}(t)$ represents the Service Pressure at node b for the microservices $m_k \in \text{Next}(f_i, m_j)$. The exponent α_{f_i} is a weight factor that differentiates microservice flows based on their priority. The value of α_{f_i} lies between 0 and 1, with a higher value representing a higher priority for the application corresponding to microservice flow f_i . This factor adjusts the weight of the service pressure, ensuring that higher priority flows are favored when allocating resources.

This computation ensures that resources (computation capacity) at the c -type node a are allocated to packets that have the largest differential backlog between a and b , taking into account the next executable microservices.

- *t-type node:* If the current node a is a t -type node (virtual transmission node), the largest differential backlog $\hat{w}_{(a,b)}$ is computed as:

$$\hat{w}_{(a,b)}(t) = \max_{f_i \in F, m_j \in M_i} \left\{ \left(\hat{Q}_{p_{f_i}}^{a m_j}(t) \right)^{\alpha_{f_i}} - \left(\hat{Q}_{p_{f_i}}^{b m_j}(t) \right)^{\alpha_{f_i}} \right\}, \quad (4)$$

where $\hat{Q}_{p_{f_i}}^{a m_j}(t)$ and $\hat{Q}_{p_{f_i}}^{b m_j}(t)$ are the Service Pressures for flow f_i associated with microservice m_j at nodes a and b , respectively. Again, α_{f_i} is used to weight the differential backlog according to the priority of the flow.

This computation ensures that transmission resources (link capacity) are allocated to packets with the largest differential backlog between a and b , thereby prioritizing flows with the greatest unmet demand at node b .

b) Backpressure-Based Routing, Resource Allocation, and Path Optimization: After calculating $\hat{w}(a, b)(t)$ for all outgoing edges (a, b) , the algorithm incorporates a path optimization mechanism to improve routing decisions. Specifically, when forwarding packets from a t -type node, the differential backlog $\hat{w}(a, b)(t)$ is adjusted by a factor $(1 + \text{Dist}(a) - \text{Dist}(b))$, where $\text{Dist}(x)$ denotes the hop distance from node x to the nearest c -type node in the same augmented graph. It is important to note that since LEO satellite orbits and their corresponding network topology are predictable, these hop distances do not require real-time pathfinding and can be efficiently pre-computed.

The path optimization mechanism addresses a common issue in traditional backpressure algorithms, which tend to exhibit high latency even under low traffic due to their reliance on maximizing differential backlogs without considering path lengths. By incorporating the hop distance factor, the algorithm ensures that packets follow the shortest available paths under low traffic conditions, thereby reducing latency. This is because when queue backlogs are minimal, the hop-distance term becomes the primary driver in the routing decision, incentivizing the selection of shorter paths. However, as the traffic load increases, the backpressure term $\hat{w}_{(a,b)}(t)$ grows to dominate the routing metric, enabling the algorithm to effectively utilize branch paths and balance network congestion.

Routing decisions are made by selecting the edge (a, b^*) with the largest adjusted metric:

$$b^* = \arg \max_{b \in \text{Neighbors}(a)} \hat{w}_{(a,b)}(t) \cdot (1 + \text{Dist}(a) - \text{Dist}(b)). \quad (5)$$

Once the outgoing edge is determined, resources (computation or transmission capacity) are allocated on the link (a, b^*) to the packets with the highest differential backlog $\hat{w}_{(a,b)}(t)$. This ensures that the network dynamically balances latency and load, improving overall performance.

c) Packet Processing and Forwarding: The algorithm handles packet forwarding differently depending on the type of virtual node. At a t -type node (virtual transport node), packets are forwarded to the selected next-hop node based on the backpressure routing and path optimization described in Step 2. No additional processing is performed at a t -type node; the packet retains its original label and routing decision.

At a c -type node (virtual compute node), packets are processed, and their execution status is updated. Specifically, after a packet is processed at a c -type node, the algorithm determines its *next executable microservice* based on the flow's dependency graph $G(n_i)$ and the packet's current microservice label. For each processed packet, the algorithm may generate multiple packet replicas (one for each next executable microservice). Each packet replica is updated with a new label corresponding to the next microservice and is routed to the next node in the augmented graph G_A^i .

More specifically, after processing a packet $p_{f_i}^{m_j}$ at a c -type node, the algorithm uses the dependency graph $G(n_i)$ to identify the next executable microservice set $M_{\text{next}} = \text{Next}(f_i, m_j)$. For each $m_k \in M_{\text{next}}$, the algorithm generates a copy of the processed packet, denoted as $p_{f_i}^{m_k}$, and forwards it to the next node in G_A^i , which must be a t -type node. (In practice, this

c -type node and its corresponding t -type node reside on the same physical satellite, so the time spent on this step can be considered negligible.)

It is crucial to emphasize that the practical implementation of Service Pressure does not require any satellite to construct or maintain the entire Augmented Sub-Graph in real-time. Rather, each satellite makes forwarding and processing decisions based on its local context within the conceptual graph. This context is determined by the microservices a satellite hosts and its physical connectivity.

To illustrate this with a concrete example, consider satellite v_9 from Fig. 2, which is configured to host microservice m_5 but not m_3 . When v_9 receives a data packet whose next required step is the execution of m_5 , it recognizes its role as a destination for computation. For this packet, it acts as a c -type node, managing its local computation queue. Conversely, if the same satellite receives a packet that ultimately requires m_3 (hosted elsewhere), it identifies itself as a transit or relay node. For this packet, it acts as a t -type node, determining the best neighboring satellite to forward the packet towards. This context-dependent, local decision-making ensures that the Service Pressure algorithm is computationally lightweight, scalable, and eminently practical for real-time operation in the dynamic LEO environment.

We summarize the Service Pressure Algorithm in Algorithm 1. This algorithm leverages the backpressure mechanism to dynamically allocate resources and determine routing paths in a distributed manner. Because routing decisions are made independently for each packet, the algorithm provides inherent flexibility, eliminating the need to invalidate or rebuild routing tables when links change. By calculating the largest differential backlog at each virtual node and forwarding packets accordingly, the algorithm ensures efficient routing and resource allocation across the augmented graph G_A^i for all microservice flows f_i .

A key advantage of the backpressure mechanism is its inherent robustness to dynamic network changes. If a link becomes temporarily unavailable for any reason, such as an ISL handover or a satellite node failure, the differential backlog towards the disconnected neighbor cannot be computed under the algorithm's standard rules. In our implementation, this value is treated as negative infinity. Consequently, the algorithm will automatically select an alternative outgoing edge with a more favorable differential backlog, effectively rerouting traffic around the failure point. This process also prevents packet loss during topology updates. If no viable outgoing edge exists, all potential links will have a differential backlog of negative infinity, and packets are securely buffered in the local backlog queue until a path becomes available. The algorithm's reliance on local queue information, rather than pre-computed global paths, allows it to react to network changes in real-time without the need for complex and time-consuming route recalculations.

In summary, the Service Pressure Algorithm provides a comprehensive solution for managing distributed microservice execution in satellite networks. It combines the throughput optimality of backpressure routing with a latency-aware path optimization mechanism. Its distributed nature not only ensures scalability but also provides strong adaptability to dynamic network conditions and link failures. By intelligently balancing

Algorithm 1: Service Pressure Algorithm With Backpressure-Based Routing.

```

1 foreach virtual node  $a \in V_A^i$  do
2   Enqueue a new packet  $p_{f_i}^{m_j}$  at time slot  $t$ :
3   if  $a$  is a  $c$ -type node then
4     foreach flow  $f_i \in F$  and microservice
        $m_j \in M_i$  do
5       Compute  $M_{\text{next}} = \text{Next}(f_i, m_j)$  from
          $G(n_i)$ ;
6       Compute Service Pressure  $\hat{Q}_{p_{f_i}}^{a_{m_j}}(t)$ ;
7       foreach  $m_k \in M_{\text{next}}$  do
8         Compute  $\hat{w}_{(a,b)}(t) =$ 
            $\max_{f_i \in F, m_j \in M_i} \{ (\hat{Q}_{p_{f_i}}^{a_{m_j}}(t))^{\alpha_{f_i}} -$ 
            $\sum_{m_k \in M_{\text{next}}} (\hat{Q}_{p_{f_i}}^{b_{m_k}}(t))^{\alpha_{f_i}} \};$ 
9         Generate packet replicas  $p_{f_i}^{m_k}$  with
           updated labels;
10        Forward  $p_{f_i}^{m_k}$  to the corresponding
           $t$ -type node in  $G_A^i$ ;
11      end
12    end
13  else if  $a$  is a  $t$ -type node then
14    foreach neighboring node  $b \in \text{Neighbors}(a)$ 
      do
15      Compute  $\hat{w}_{(a,b)}(t) =$ 
         $\max_{f_i \in F, m_j \in M_i} [\hat{Q}_{p_{f_i}}^{a_{m_j}}(t) - \hat{Q}_{p_{f_i}}^{b_{m_j}}(t)]$ ;
16      Adjust  $\hat{w}_{(a,b)}(t)$  by path optimization factor
         $(1 + \text{Dist}(a) - \text{Dist}(b))$ ;
17    end
18    Select next-hop
       $b^* = \arg \max_{b \in \text{Neighbors}(a)} \hat{w}_{(a,b)}(t) \cdot (1 +$ 
       $\text{Dist}(a) - \text{Dist}(b))$ ;
19    Forward the packet to  $b^*$  based on computed
      backpressure and available link capacity;
20  end
21 end

```

throughput, delay, and network stability, our approach offers significant advantages for high-performance, next-generation satellite network scenarios.

d) Overhead and Scalability Analysis: The Service Pressure Algorithm has low and localized operational overhead, making it well-suited for real-world deployment. This can be analyzed in terms of communication and computation:

- **Communication Overhead and Scalability:** The algorithm avoids any form of global state dissemination. The only required communication is the exchange of all local backlog queues' length between immediate neighbors once per time slot. This localized approach defines its scalability. The overhead per satellite is $O(1)$ with respect to the constellation, as the number of neighbors is a small constant. The size of the exchanged message, however, scales linearly with the total number of service states. This complexity

is $O(|N| \times l)$, where $|N|$ is the number of microservice flows and l is the average number of services per flow. Since the flow length l is typically a bounded constant for real-world applications, the overhead's growth is primarily driven by the number of flows $|N|$. This is a practical and manageable trade-off.

- **Computational and State Overhead:** The real-time computational load on each satellite is negligible. The hop distance metric can be pre-computed and stored. The real-time decision for each packet thus reduces to a few simple arithmetic operations: calculating the differential backlog for each of its neighbors, applying the pre-computed metric, and selecting the maximum. Consequently, the computational complexity per decision is $O(1)$. The memory required to maintain the backlog queues scales as $O(|N| \times l)$. This lightweight process ensures that routing decisions can be made well within a short time slot without straining the satellite's processing resources.

In summary, the Service Pressure Algorithm is highly scalable for growing constellations and maintains a practical, linear overhead for an increasing number of services, confirming its suitability for complex, real-world deployments.

V. EXPERIMENTAL EVALUATION

A. Experiment Setup

We implement a prototype of the Service Pressure Algorithm on a 72/12/1 Telesat Star Constellation system, which consists of 6 polar orbital planes with 12 satellites in each plane [35]. Each satellite is equipped with four ISLs, including two intra-plane ISLs that connect satellites within the same orbital plane and two inter-plane ISLs that connect satellites in adjacent orbital planes. According to [35], the Telesat constellation operates at an orbital altitude of 1000 km with an inclination angle of 99.5° , ensuring near-global coverage, particularly in high-latitude regions. The inter-satellite distance between neighboring satellites is approximately over 3000 km. The duration of the time slot is set to 10 milliseconds, aligned with the network's characteristics. During this short time slot, the topology of satellite network can be assumed static. Furthermore, this duration matches the network's characteristic propagation delay, which ensures that the queue backlog information used for backpressure calculations remains timely without incurring excessive signaling overhead.

Our experimental focus is on a dynamic and diverse microservice environment. We randomly deploy $|\mathcal{M}| = 60$ distinct types of microservices across the constellation. Each microservice has between 3 and 8 replicas distributed uniformly, with up to 6 replicas on any single satellite. Additionally, we configure 4 different applications, each invoking a randomly generated dependency graph of 8 to 10 microservices in sequence. The data for each flow is packetized into 512B packets. Based on [36], the computational capacity of each satellite is drawn from a uniform distribution between 100,000 and 150,000 microservice packets per second. To create a fair and unbiased evaluation environment for this randomized service deployment, we adopt a homogeneous capacity allocation for the underlying network, a common practice in LEO network studies [34]. Specifically,

each ISL is assigned a uniform transmission capacity of 30Mbps at the start of the experiment, a moderate and practical data rate for current ISL technology [13]. This standardized capacity provided a consistent baseline, ensuring that the performance of any randomly generated service flow was not arbitrarily distorted by the heterogeneous physical layer.

B. Evaluation Methodology

1) *Baseline Methods*: To evaluate the performance of the proposed service stress algorithm, we compare it with five baseline methods: RMPR, LCRA, QueueFlow, Dynamic A* and SR. Given that some of the baselines do not consider microservice dependencies, we make basic modifications to adapt to the experimental environment. Specifically, a naive greedy strategy is adopted to select the nearest satellite that can host the next required microservice in the dependency graph.

a) *RMPR (Request Matching and Partition Mapping Routing)*: RMPR [25] is a two-stage heuristic algorithm that probabilistically splits request flows between microservice instances through request matching and partition mapping algorithms, thereby balancing response delays and ensuring high request success rates.

b) *LCRA (Low Complexity Routing Algorithm)*: LCRA [15] is a low-complexity routing algorithm designed for routing in LEO satellite networks. It utilizes a distributed approach based on satellite position and dynamic congestion information to dynamically select the next hop for packets.

c) *QueueFlow*: QueueFlow (QF) [26] is a dynamic queue balancing framework for orchestrating microservice workflows, leveraging real-time latency feedback to estimate queue lengths and dynamically allocate routing paths and computing resources among services. QueueFlow is particularly effective in handling non-stationary workloads because it can dynamically adjust resource allocation without relying on offline dependency analysis.

d) *Dynamic A**: Dynamic A* [29] is a hybrid dynamic routing algorithm that integrates the A* search algorithm with an Ant Colony Optimization (ACO) framework for LEO satellite networks. It uses the A* algorithm for efficient, heuristic-based pathfinding, while leveraging the ACO mechanism, where probe ant packets deposit pheromones, to dynamically sense and update link costs based on real-time network congestion. This allows the algorithm to find low-latency paths while adaptively avoiding congested nodes and links.

e) *SR (Stable Route)*: SR [27] is a routing algorithm based on Dijkstra, specifically adapted for dynamic LEO satellite networks. Its primary contribution is the introduction of a link stability factor that incorporates both transmission delay and the link's remaining lifetime into the path cost. This approach prioritizes more stable, longer-lasting paths, thereby reducing route flapping and the overhead associated with frequent recalculations in a constantly changing topology.

2) *Evaluation Metrics*: To comprehensively evaluate the performance of the proposed Service Pressure Algorithm and compare it with the baseline methods, we adopt the following three metrics:

a) *Request Completion Rate*: This metric measures the ratio of successfully completed requests to the total number of sent requests. A higher request completion rate indicates better system reliability and effectiveness in handling service requests under varying workloads.

b) *Average Latency*: This metric captures the average time, measured in time slots, required to complete a full service request. It accounts for all stages of the request's lifecycle, including queuing, computation, and transmission delays. Lower average latency reflects the system's ability to efficiently process service requests and minimize delays.

c) *P90 Tail Latency*: The P90 tail latency, or the 90th percentile latency, represents the latency below which 90% of requests are completed. It provides a measure of the worst-case latency experienced by the majority of requests and highlights the system's ability to handle high-load scenarios without significant delays. Lower P90 tail latency indicates better performance in maintaining service quality under peak traffic conditions.

C. Experimental Results

1) *Poisson Request Arrival*: We vary the average request arrival rate from 24.576 Mbps to 49.152 Mbps, with an interval of 4.096 Mbps. As illustrated in Fig. 3(a), Service Pressure achieves the highest completion ratio across all tested arrival rates, maintaining an almost perfect completion ratio ($> 99\%$) up to a Poisson arrival rate of 40.960 Mbps. Even at the highest arrival rate of 49.152 Mbps, the completion ratio remains at 95.96%. This performance demonstrates the robustness of Service Pressure in handling high traffic loads, owing to its ability to dynamically allocate resources based on queue backlogs. In comparison, QF also achieves high completion ratios for lower arrival rates, but its performance degrades significantly under higher loads, dropping to 53.86% at 49.152 Mbps. LCRA exhibits a steady but relatively lower completion ratio, starting at 86.87% for the lowest arrival rate and declining to 42.19% at 49.152 Mbps. RMPR, which prioritizes load balancing via request matching, struggles the most under high traffic, with a steep decline from 70.76% to 21.88%. Dynamic A* and SR algorithms exhibit similar completion rates to LCRA, both decreasing with increasing request arrival rates.

We plot the average latency in Fig. 3(b), where the latency unit is time slot, and each time slot is 10 ms. Service Pressure demonstrates the lowest average latency across all arrival rates, starting at 37.27 time slots for 24.576 Mbps and gradually increasing to 107.50 time slots at 49.152 Mbps. In comparison, QF exhibits competitive latency for lower traffic but experiences sharp increases at higher loads, reaching 213.47 time slots at 49.152 Mbps. LCRA maintains relatively consistent latencies (around 100 time slots) due to its reliance on fixed routing paths, but this comes at the cost of completion ratio and throughput. RMPR suffers from higher latencies throughout, starting at 75.65 time slots and rising to 113.02 time slots at the highest arrival rate, due to its inability to dynamically adjust to congestion. Fig. 3(c) shows the P90 tail latency of all algorithms, Service Pressure outperforms all baselines with consistently lower tail latency, starting at 46.76 time slots and increasing to 158.29

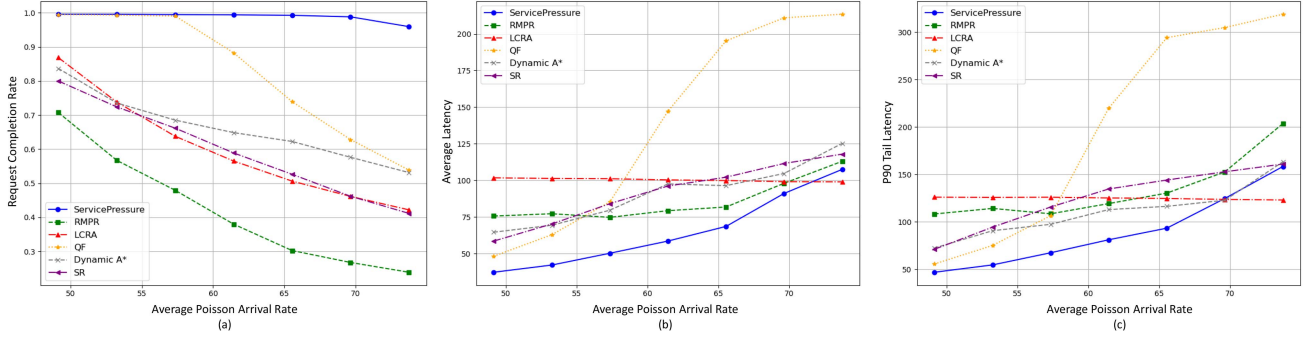


Fig. 3. The performance under Poisson request arrival rate. (a) Request completion rate, (b) Average latency, (c) P90 tail latency.

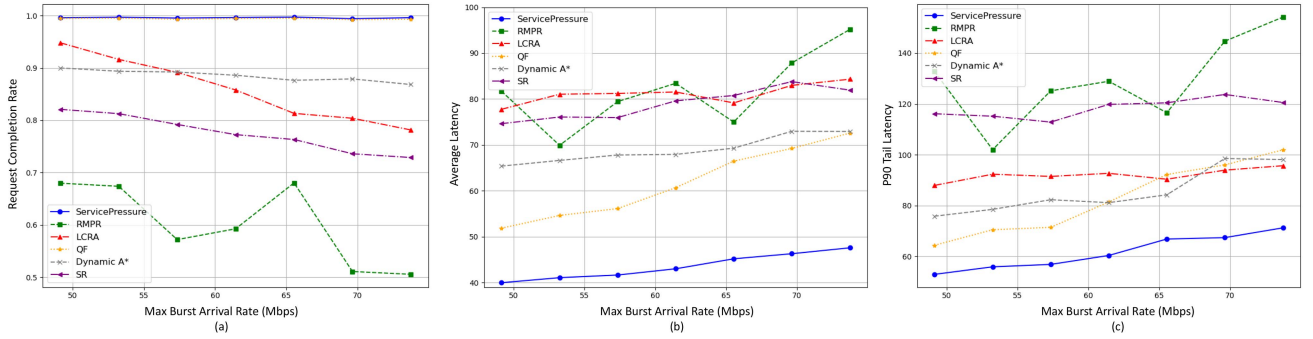


Fig. 4. The performance under burst request arrival rate. (a) Request completion rate, (b) Average latency, (c) P90 Tail latency.

time slots at 49.152 Mbps. In comparison, QF shows acceptable tail latency at lower arrival rates but rapidly degrades under higher traffic, reaching 318.89 time slots at 49.152 Mbps. LCRA maintains stable tail latency (around 125 time slots), reflecting its deterministic routing strategy but limited adaptability. RMPR performs poorly, with P90 latency climbing from 108.21 time slots to 203.68 time slots, highlighting its limitations in handling higher loads efficiently. Dynamic A* and SR algorithms are designed for end-to-end data transmission. They exhibited decent latency at lower arrival rates, but both experienced significant latency increases as request arrival rates increased. This is primarily due to their lack of a microservice-specific scheduling mechanism, which makes computation nodes more susceptible to congestion.

In summary, the results clearly show that Service Pressure has advantages in terms of completion ratio, average latency, and tail latency under all tested traffic conditions. The results highlight its ability to dynamically balance traffic and effectively allocate computing resources and routing paths, making it robust under both low and high arrival rates. In contrast, QF struggles to maintain performance under high load, while LCRA and RMPR fall short in both adaptability and data delivery performance, Dynamic A* and SR lack the ability to schedule and allocate computing resources for microservices.

2) *Burst Request Arrival*: We further evaluate the performance of all algorithms under a burst traffic pattern, where each transmission period alternates between a normal phase and a burst phase. The normal phase lasts an average of 2 seconds with a constant bit rate (CBR) of 20.48 Mbps, while the burst phase

lasts 1 s with a randomly selected transmission rate between 20.48 Mbps and the maximum burst rate. The results are shown in Fig. 4.

As illustrated in Fig. 4(a), Service Pressure maintains an almost perfect completion ratio across all tested burst rates, consistently exceeding 99% even under the maximum burst rate of 73.728 Mbps. This demonstrates the algorithm's strong adaptability to high burst traffic, which is attributed to its ability to dynamically allocate resources based on queue backlogs, ensuring high throughput and reliability. Similarly, QF also achieves high completion ratios, benefiting from its backlog-queue-like resource allocation. While its performance slightly lags behind Service Pressure, it still maintains a completion ratio above 99% at all tested rates. In contrast, LCRA, RMPR, Dynamic A* and SR exhibit significantly lower completion ratios. RMPR, which employs load balancing via request matching, struggles the most, highlighting its inability to efficiently handle highly dynamic traffic.

The average latency results are shown in Fig. 4(b). Service Pressure achieves the lowest average latency across all burst rates, starting at 39.97 time slots for 49.152 Mbps and gradually increasing to 47.58 time slots at 73.728 Mbps. This low and steady latency indicates its ability to handle bursty traffic without significant delays. QF shows slightly higher latencies than Service Pressure, starting at 51.85 time slots and increasing to 72.56 time slots as the burst rate increases. In contrast, LCRA and SR maintain relatively consistent but higher latencies, averaging around 80 time slots, which stems from their stable routing strategy that limits its ability to minimize delays under fluctuating

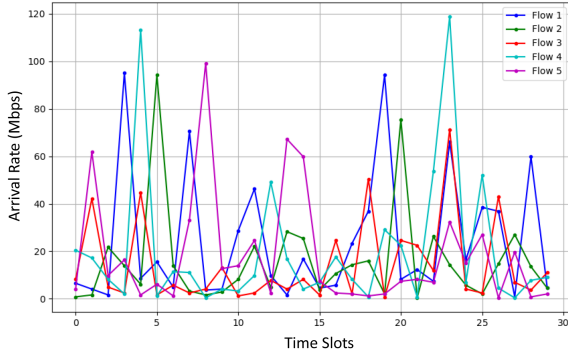


Fig. 5. The request arrival pattern of a non-stationary traffic.

traffic. Dynamic A* is better able to perceive link congestion in real time, maintaining an average latency of approximately 70 time slots. RMPR suffers from the highest latencies, starting at 81.69 time slots at 49.152 Mbps and rising to 95.15 time slots at 73.728 Mbps, indicating its inability to adapt to sudden bursts.

The P90 tail latency results are presented in Fig. 4(c). Service Pressure outperforms all baselines, maintaining the lowest tail latency across all burst rates. It starts at 52.92 time slots for 49.152 Mbps and rises to 71.22 time slots at 73.728 Mbps, demonstrating its robustness in mitigating long-tail effects even under high traffic bursts. QF and Dynamic A* also exhibit competitive tail latencies, though slightly higher than Service Pressure. LCRA and SR exhibit stable but higher tail latencies, with the latter relatively higher. RMPR performs the worst, with P90 latency increasing from 132.74 time slots to 154.18 time slots, highlighting its inefficiency in managing tail delays under burst traffic conditions.

In summary, Service Pressure consistently demonstrates superior performance across all metrics under burst traffic patterns. Its dynamic queue-backlog-driven resource allocation enables it to achieve high throughput, low latency, and robust tail latency control, even at maximum burst rates. QF, leveraging a similar backlog-based design, also performs well, though its latency and tail latency are slightly higher. In contrast, other baselines fail to adapt to the dynamic nature of burst traffic, resulting in lower completion ratios and higher latencies.

3) *Non-Stationary Request Arrival*: We evaluate Service Pressure and other baseline algorithms under a non-stationary request arrival pattern, where the arrival rate fluctuates dynamically over time. The data used for this evaluation is sampled from a real-world workload pattern extracted from a production MLaaS (Machine Learning as a Service) cluster in Alibaba. From this dataset, we randomly selected five microservice flows and sampled the request call volume over a specific period. Fig. 5 shows part of the obtained sampling data. The sampled data captures the irregularity and variability typical of real-world traffic patterns, where the request arrival rate varies according to the specific usage and load conditions of the microservices.

Table I shows the performance differences of Service Pressure and other baselines under non-stationary arrival patterns. As shown in the results, the non-stationary nature of the traffic introduces additional challenges in microservice scheduling

TABLE I
COMPARISON UNDER NON-STATIONARY REQUEST ARRIVAL

Metrics	Throughput	Completion Ratio	Average Latency	P90 Tail Latency
ServicePressure	119.96 Mbps	97.61%	502.917 ms	626.337 ms
RMPR	53.97 Mbps	44.18%	801.523 ms	995.526 ms
LCRA	84.02 Mbps	68.36%	915.649 ms	984.203 ms
QF	108.13 Mbps	88.55%	1234.426 ms	1621.611 ms
Dynamic A*	100.24 Mbps	81.56%	859.975 ms	1000.379 ms
SR	67.08 Mbps	54.61%	1164.480 ms	1461.283 ms

and routing. Service Pressure demonstrates its robustness in handling such non-stationary traffic by adapting dynamically to changing request patterns, outperforming the baseline algorithms in the non-stationary traffic scenario, highlighting its adaptability and efficiency in dynamically managing resources. The ability to adjust in real-time to varying request patterns enables Service Pressure to provide superior performance in highly dynamic environments, making it a promising solution for microservice systems operating in real-world environments with unpredictable traffic patterns.

4) *Effect of Weight Factor on Service Priority*: In this experiment, we investigate the effect of adjusting the weight factor α_f on the priority of different microservice flows. We set up 8 microservice flows, with half representing high-priority applications and the other half representing low-priority applications. For high-priority applications, the weight factor α_f is fixed at 1, while we adjust the weight factor for low-priority applications. The experimental results shown in Fig. 6 demonstrate that adjusting the weight factor α_f allows for effective prioritization of high-priority applications. By reducing the weight of low-priority applications, the system allocates more resources to high-priority flows, improving their completion rate and reducing latency. However, this comes at the cost of lower performance for low-priority applications, highlighting the trade-off between prioritizing different service flows. These results validate the flexibility and effectiveness of the weight adjustment mechanism in controlling service flow priorities within our system.

5) *Impact of Faulty Satellites and Link Failures*: To further validate the robustness of Service Pressure against the dynamic and challenging conditions of LEO satellite networks, we conducted an experiment simulating satellite failures. In a real-world scenario, satellites may temporarily cease their primary functions due to factors like limited power supply (entering a low-power mode), hardware malfunctions, or other unexpected faults. In our simulation, we model such an event by defining a “faulty” satellite as a node that suspends all microservice processing capabilities, acting solely as a data packet relay. Additionally, the ISLs connected to this faulty satellite may experience random disconnections. The experiment was conducted on the 72/12/1 Telesat constellation, where satellite failures could occur at any time with a variable duration. However, the total number of faulty satellites across the constellation at any given moment was maintained at a constant percentage. We evaluated the performance of all algorithms under scenarios where 0%, 5%, 10%, 15%, 20%, and 25% of the satellites were in a faulty state. The average request arrival rate of all flows is fixed at 32.768 Mbps.

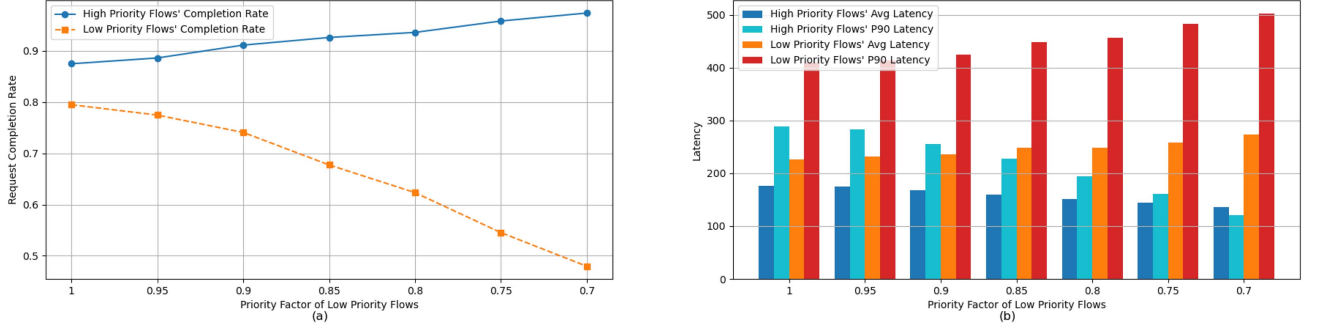


Fig. 6. The impact of different priority settings on service routing. (a) Request completion rate, (b) Latency.

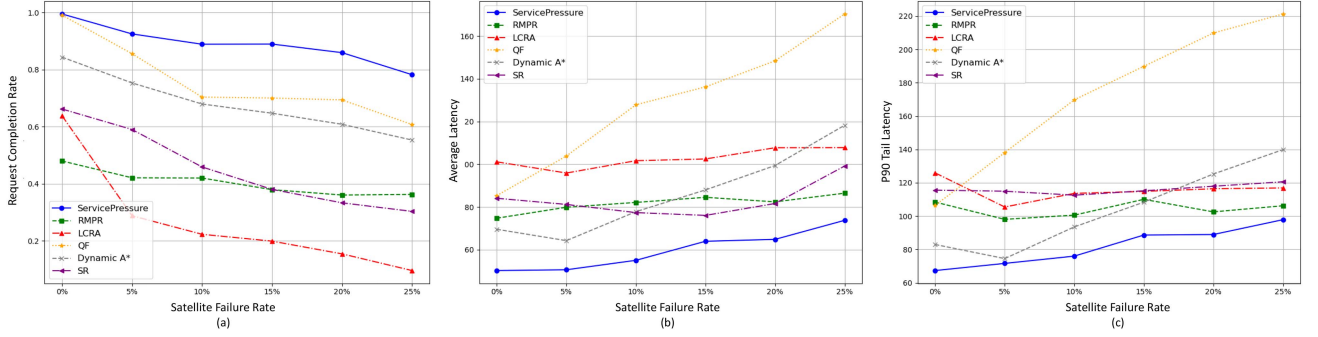


Fig. 7. The impact of faulty satellites and link failures on service routing. (a) Request completion rate, (b) Average latency, (c) P90 tail latency.

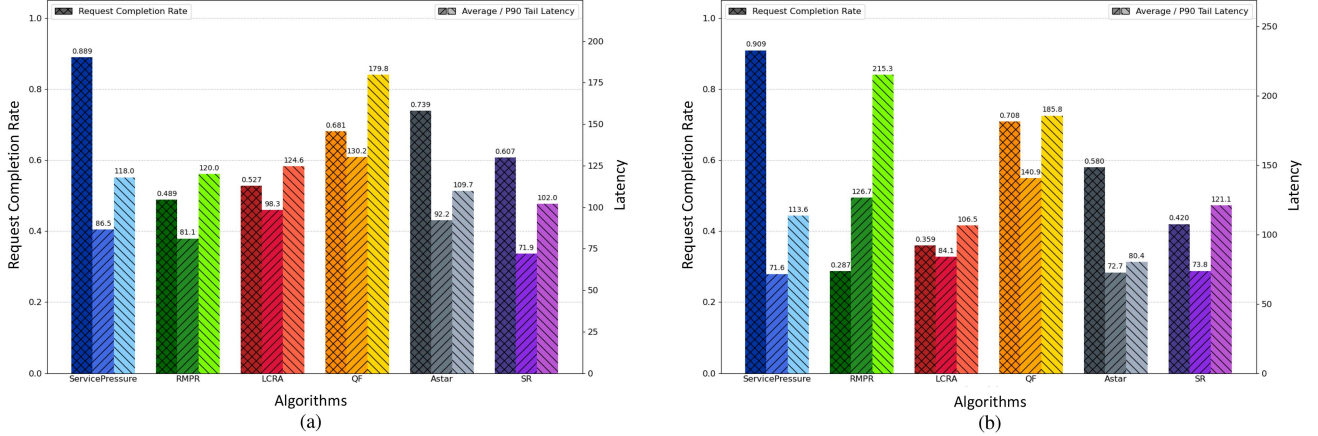


Fig. 8. Performance comparison of different routing algorithms across two distinct constellation architectures: (a) The iridium-like constellation. (b) The telesat inclined orbit constellation. For each sub-figure, the left Y-axis represents the request completion rate, while the right Y-axis represents average and P90 tail latency.

As depicted in Fig. 7, the increasing prevalence of faulty satellites understandably degrades the performance of all evaluated algorithms, leading to a lower request completion rate and increased latency. However, the results clearly demonstrate that Service Pressure exhibits significantly higher resilience compared to the baseline methods. Even with 25% of the satellites operating in a faulty state, Service Pressure maintains a superior request completion rate and effectively contains the growth in latency. This superior performance is attributed to its distributed, backpressure-based nature, as analyzed in Section IV-B. When

a satellite stops processing microservices, the algorithm inherently detects the rapidly accumulating queues for services hosted on that node. Consequently, it automatically reroutes subsequent service requests to healthy, alternative satellite replicas elsewhere in the network. Similarly, when ISLs connected to a faulty node fail, the algorithm instantly bypasses these broken links by selecting other viable paths based on local queue pressure. This adaptive rerouting prevents catastrophic packet loss and avoids the long delays associated with route reconvergence that affect less dynamic algorithms.

6) *Performance Across Different Constellation Structures:* To evaluate the generalizability and robustness of our proposed algorithm, we extended our experiments to two additional, distinct constellation architectures. The first is a classic Iridium-like polar constellation, comprising 66 satellites distributed across 6 orbital planes with 11 satellites per plane [37]. The second is the Telesat inclined orbit constellation [35]. Unlike its polar counterpart, it is specifically designed to provide concentrated coverage over high-traffic mid-latitude regions and thus exhibits different topological dynamics. For consistency, the average request arrival rate of all flows is fixed at 32.768 Mbps.

The performance comparison of the algorithms in these two new constellations is presented in Fig. 8. The results consistently demonstrate the superior performance of Service Pressure across both distinct architectural models. In both the Iridium-like (Fig. 8(a)) and the Telesat inclined orbit (Fig. 8(b)) scenarios, Service Pressure achieves the highest request completion rate by a significant margin, underscoring its ability to effectively manage microservice flows regardless of the underlying network structure. While maintaining high rate of successful request completion, its average and P90 tail latencies remain highly competitive, often outperforming other high-throughput methods like QF. This performance is a direct result of the algorithm's core design principles. The backpressure mechanism effectively adapts to the unique link dynamics of each constellation to maximize throughput, while the path optimization mechanism ensures that this does not come at the cost of excessive delays. This validates that the effectiveness of Service Pressure is not confined to a single topology but is a fundamental property of its adaptive routing strategy.

VI. CONCLUSION

In this paper, we propose Service Pressure, a dynamic distributed method designed to optimize microservice workflows in satellite networks. Our approach leverages satellite constellation topology information and microservice deployment data to construct an Augmented Satellite-Microservice Flow Sub-Graph. By running a path-optimized backpressure algorithm on this graph, we can efficiently route service requests, addressing the unique challenges posed by satellite networks, such as high latency, limited bandwidth, and dynamic topologies. Extensive experiments demonstrated that the Service Pressure algorithm outperforms other baseline methods in various traffic patterns, particularly excelling at handling high traffic volumes, and can flexibly control the priority of different requests. These results highlight the algorithm's robust adaptability and effectiveness in real-world satellite network environments. Given the increasing reliance on satellite networks for global communication and computation, the ability of Service Pressure to optimize microservice routing and scheduling makes it an ideal solution for the next-generation satellite-based microservice architectures.

REFERENCES

- [1] S. Ma, Y. C. Chou, H. Zhao, L. Chen, X. Ma, and J. Liu, "Network characteristics of LEO satellite constellations: A starlink-based measurement from end users," in *Proc. IEEE Conf. Comput. Commun.*, 2023, pp. 1–10.
- [2] W. Mi and Y. Fang, "Intelligent remote sensing satellite and remote sensing image real-time service," *Acta Geodetica et Cartographica Sinica*, vol. 48, no. 12, 2019, Art. no. 1586.
- [3] Z. Shen et al., "A survey of next-generation computing technologies in space-air-ground integrated networks," *ACM Comput. Surv.*, vol. 56, no. 1, pp. 1–40, 2023.
- [4] I. Leyva-Mayorga et al., "Satellite edge computing for real-time and very-high resolution earth observation," *IEEE Trans. Commun.*, vol. 71, no. 10, pp. 6180–6194, Oct. 2023.
- [5] B. Denby and B. Lucia, "Orbital edge computing: Machine inference in space," *IEEE Comput. Archit. Lett.*, vol. 18, no. 1, pp. 59–62, Jan.–Jun. 2019.
- [6] T.-A. Bui, P.-J. Lee, K.-Y. Chen, C.-R. Chen, C. S. Liu, and H.-C. Lin, "Edge computing-based sat-video coding for remote sensing," *IEEE Access*, vol. 10, pp. 52840–52852, 2022.
- [7] M. Fazio, A. Celesti, R. Ranjan, C. Liu, L. Chen, and M. Villari, "Open issues in scheduling microservices in the cloud," *IEEE Cloud Comput.*, vol. 3, no. 5, pp. 81–88, Sep./Oct. 2016.
- [8] N. Alshuqayran, N. Ali, and R. Evans, "A systematic mapping study in microservice architecture," in *Proc. IEEE 9th Int. Conf. Serv.-Oriented Comput. Appl.*, 2016, pp. 44–51.
- [9] W. Zhang, M. Cai, T. Zhang, Y. Zhuang, and X. Mao, "EarthGPT: A universal multi-modal large language model for multi-sensor image comprehension in remote sensing domain," *IEEE Trans. Geosci. Remote Sens.*, vol. 62, 2024, Art. no. 5917820.
- [10] Y. Wang et al., "MPCSM: Microservice placement for edge-cloud collaborative smart manufacturing," *IEEE Trans. Ind. Informat.*, vol. 17, no. 9, pp. 5898–5908, Sep. 2021.
- [11] S. Y. Jang, B. Kostadinov, and D. Lee, "Microservice-based edge device architecture for video analytics," in *Proc. IEEE/ACM Symp. Edge Comput.*, 2021, pp. 165–177.
- [12] Z. Xiao et al., "LEO satellite access network (LEO-SAN) towards 6G: Challenges and approaches," *IEEE Wireless Commun.*, vol. 31, no. 2, pp. 89–96, Apr. 2024.
- [13] I. Leyva-Mayorga, B. Soret, and P. Popovski, "Inter-plane inter-satellite connectivity in dense LEO constellations," *IEEE Trans. Wireless Commun.*, vol. 20, no. 6, pp. 3430–3443, Jun. 2021.
- [14] Y. Chen et al., "Unraveling physical space limits for LEO network scalability," in *Proc. 23rd ACM Workshop Hot Topics Netw.*, 2024, pp. 43–51.
- [15] X. Liu, X. Yan, Z. Jiang, C. Li, and Y. Yang, "A low-complexity routing algorithm based on load balancing for LEO satellite networks," in *Proc. IEEE 82nd Veh. Technol. Conf.*, 2015, pp. 1–5.
- [16] X. Li, F. Tang, L. Chen, and J. Li, "A state-aware and load-balanced routing model for LEO satellite networks," in *Proc. IEEE Glob. Commun. Conf.*, 2017, pp. 1–6.
- [17] F. Tang, H. Zhang, and L. T. Yang, "Multipath cooperative routing with efficient acknowledgement for LEO satellite networks," *IEEE Trans. Mobile Comput.*, vol. 18, no. 1, pp. 179–192, Jan. 2019.
- [18] S. Moeller, A. Sridharan, B. Krishnamachari, and O. Gnawali, "Routing without routes: The backpressure collection protocol," in *Proc. 9th ACM/IEEE Int. Conf. Inf. Process. Sensor Netw.*, 2010, pp. 279–290.
- [19] J. Ryu, L. Ying, and S. Shakkottai, "Back-pressure routing for intermittently connected networks," in *Proc. IEEE INFOCOM*, 2010, pp. 1–5.
- [20] A. Dvir and A. V. Vasilakos, "Backpressure-based routing protocol for DTNs," in *Proc. ACM SIGCOMM 2010 Conf.*, 2010, pp. 405–406.
- [21] A. Ruiz-Zafra, J. Pigueiras-del Real, M. Noguera, L. Chung, D. G. Barres, and K. Benghazi, "Servitization of customized 3D assets and performance comparison of services and microservices implementations," *IEEE Trans. Serv. Comput.*, vol. 17, no. 1, pp. 194–208, Jan./Feb. 2024.
- [22] S. Ye et al., "Galaxy: A resource-efficient collaborative edge ai system for in-situ transformer inference," 2024, *arXiv:2405.17245*.
- [23] L. Bao, C. Wu, X. Bu, N. Ren, and M. Shen, "Performance modeling and workflow scheduling of microservice-based applications in clouds," *IEEE Trans. Parallel Distrib. Syst.*, vol. 30, no. 9, pp. 2114–2129, Sep. 2019.
- [24] A. Samanta and J. Tang, "Dyme: Dynamic microservice scheduling in edge computing enabled IoT," *IEEE Internet Things J.*, vol. 7, no. 7, pp. 6164–6174, Jul. 2020.
- [25] Y. Hu, H. Wang, L. Wang, M. Hu, K. Peng, and B. Veeravalli, "Joint deployment and request routing for microservice call graphs in data centers," *IEEE Trans. Parallel Distrib. Syst.*, vol. 34, no. 11, pp. 2994–3011, Nov. 2023.
- [26] H. Cao, X. Liu, H. Guo, J. He, and X. Liu, "Queueflower: Orchestrating microservice workflows via dynamic queue balancing," in *Proc. IEEE Int. Conf. Web Serv.*, 2024, pp. 1293–1299.

- [27] T. Pan et al., “Stableroute: When Dijkstra’s algorithm meets topology-varying satellite networks,” in *Proc. IEEE Conf. Comput. Commun.*, 2025, pp. 1–10.
- [28] S. Mao, Y. Ouyang, C. Yang, and Z. Wang, “Intent-driven segment routing design for large-scale LEO satellite networks,” in *Proc. Int. Wireless Commun. Mobile Comput.*, 2025, pp. 138–143.
- [29] Z. Niu, Y. Xie, Z. Li, and J. Fu, “LEO satellite routing method based on dynamic A-star algorithm,” in *Proc. IEEE Int. Conf. Artif. Intell. Power Syst.*, 2024, pp. 233–237.
- [30] Y. Huang and X. Zhang, “Microservice scheduling for satellite-terrestrial hybrid network with edge computing,” in *Proc. IEEE/CIC Int. Conf. Commun. China*, 2022, pp. 24–29.
- [31] Y. Huang, X. Zhang, and Z. Xu, “SatEdge: Platform of edge cloud at satellite and scheduling mechanism for microservice modules,” *IEEE Access*, vol. 11, pp. 126283–126294, 2023.
- [32] X. Lan, Y. Chen, and L. Cai, “Throughput-optimal H-QMW scheduling for hybrid wireless networks with persistent and dynamic flows,” *IEEE Trans. Wireless Commun.*, vol. 19, no. 2, pp. 1182–1195, Feb. 2020.
- [33] J. Chen, L. Liu, and X. Hu, “Towards a throughput-optimal routing algorithm for data collection on satellite networks,” *Int. J. Distrib. Sensor Netw.*, vol. 12, no. 7, 2016, Art. no. 1550147716658608, doi: [10.1177/1550147716658608](https://doi.org/10.1177/1550147716658608).
- [34] X. Deng, L. Chang, S. Zeng, L. Cai, and J. Pan, “Distance-based backpressure routing for load-balancing LEO satellite networks,” *IEEE Trans. Veh. Technol.*, vol. 72, no. 1, pp. 1240–1253, Jan. 2023.
- [35] I. Del Portillo, B. G. Cameron, and E. F. Crawley, “A technical comparison of three low earth orbit satellite constellation systems to provide global broadband,” *Acta Astronautica*, vol. 159, pp. 123–135, 2019.
- [36] J. Cao, S. Zhang, Q. Chen, H. Wang, M. Wang, and N. Liu, “Computing-aware routing for LEO satellite networks: A transmission and computation integration approach,” *IEEE Trans. Veh. Technol.*, vol. 72, no. 12, pp. 16607–16623, Dec. 2023.
- [37] C. Fossa, R. Raines, G. Gunsch, and M. Temple, “An overview of the iridium (R) low earth orbit (LEO) satellite system,” in *Proc. IEEE Nat. Aerosp. Electron. Conf.*, 1998, pp. 152–159.



Xindi He (Graduate Student Member, IEEE) received the BS degree in software engineering from East China Normal University, China, in 2024. He is currently working toward the MS degree with the Software Engineering Institute, East China Normal University, China. His research interests include intelligent networking, machine learning, and deep reinforcement learning.



Ting Wang (Senior Member, IEEE) received the PhD degree in computer science and engineering from the Hong Kong University of Science and Technology, Hong Kong, China, in 2015. He is currently an associate professor with the Shanghai Key Laboratory of Trustworthy Computing; the MoE Engineering Research Center of Software/Hardware Co-design Technology and Application; East China Normal University (ECNU), Shanghai, China. Prior to joining ECNU, in 2020, he worked at the Bell Labs as a research scientist from 2015 to 2016, and with Huawei as a senior engineer from 2016 to 2020. He served as an associate editor of *IEEE Access*, the founding editor-in-chief of *IITCIB*, and the technical committee member of *Computer Communications*, Elsevier. His research interests include machine learning, edge AI, AI-aided intelligent networking, and cloud/edge computing.



Yuanming Shi (Senior Member, IEEE) received the BS degree in electronic engineering from Tsinghua University, Beijing, China, in 2011, and the PhD degree in electronic and computer engineering from The Hong Kong University of Science and Technology (HKUST), in 2015. Since 2015, he has been with the School of Information Science and Technology in ShanghaiTech University, where he is a full professor. He visited University of California, Berkeley, CA, USA, from 2016 to 2017. His research areas include edge artificial intelligence and large-scale optimization. He is a recipient of the IEEE Marconi Prize Paper Award in Wireless Communications, in 2016, the Young Author Best Paper Award by the IEEE Signal Processing Society in 2016, the IEEE ComSoc Asia-Pacific Outstanding Young Researcher Award, in 2021, the Chinese Institute of Electronics First Prize in Natural Science, in 2022, and the China Institute of Communications First Prize in Natural Science, in 2024. He is also an editor of *IEEE Transactions on Wireless Communications*, *IEEE Journal on Selected Areas in Communications*, *Journal of Communications and Information Networks*, and *Space Habitation*. He is an IET fellow.



Xin Liu (Member, IEEE) received the PhD degree in electrical engineering from Arizona State University. He was a post-doctoral research fellow with the Department of Electrical Engineering and Computer Science, University of Michigan, Ann Arbor. He is currently a Tenure-Track assistant professor with the School of Information Science and Technology, ShanghaiTech University. His research interests lie broadly in stochastic modeling, analysis, and optimization, online learning and decision-making, and reinforcement learning, with applications in large-scale server systems, communication, and ride-sharing networks.